



Mobile Devices and Applications

Unit 6 – Adapters & Lists

Summary

Adaptors

- ArrayAdapter, SimpleAdapter, BaseAdapter

Lists

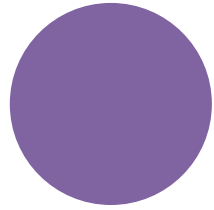
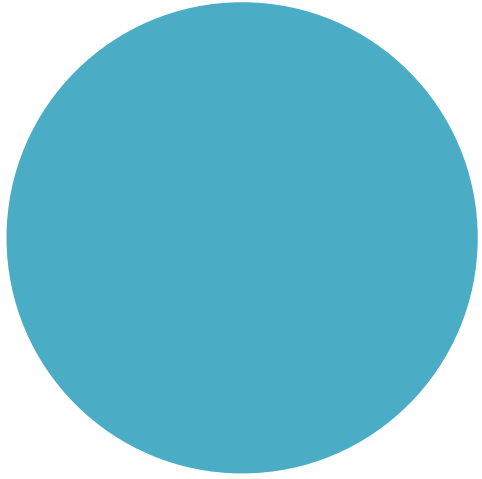
- ListView, Spinner, GridView, etc.

Selection event

- Event handlers

Serialization

- Parcelable implementation



COMPLEX VISUAL CONTROLS



Complex visual controls

- **ListView, GridView, Spinner**
 - Complex controls which implement the abstract class **AdapterView**
- An adaptor is needed for initialization
 - Classes that extend **BaseAdapter** (which implements the **Adapter** interface):
 - **ArrayAdapter**
 - **CursorAdapter**
 - **SimpleCursorAdapter**
 - **SimpleAdapter**
- Binding: by the use of the **setAdapter()** method

Other usages

AutoCompleteTextView

MultiAutoCompleteTextView

Gallery

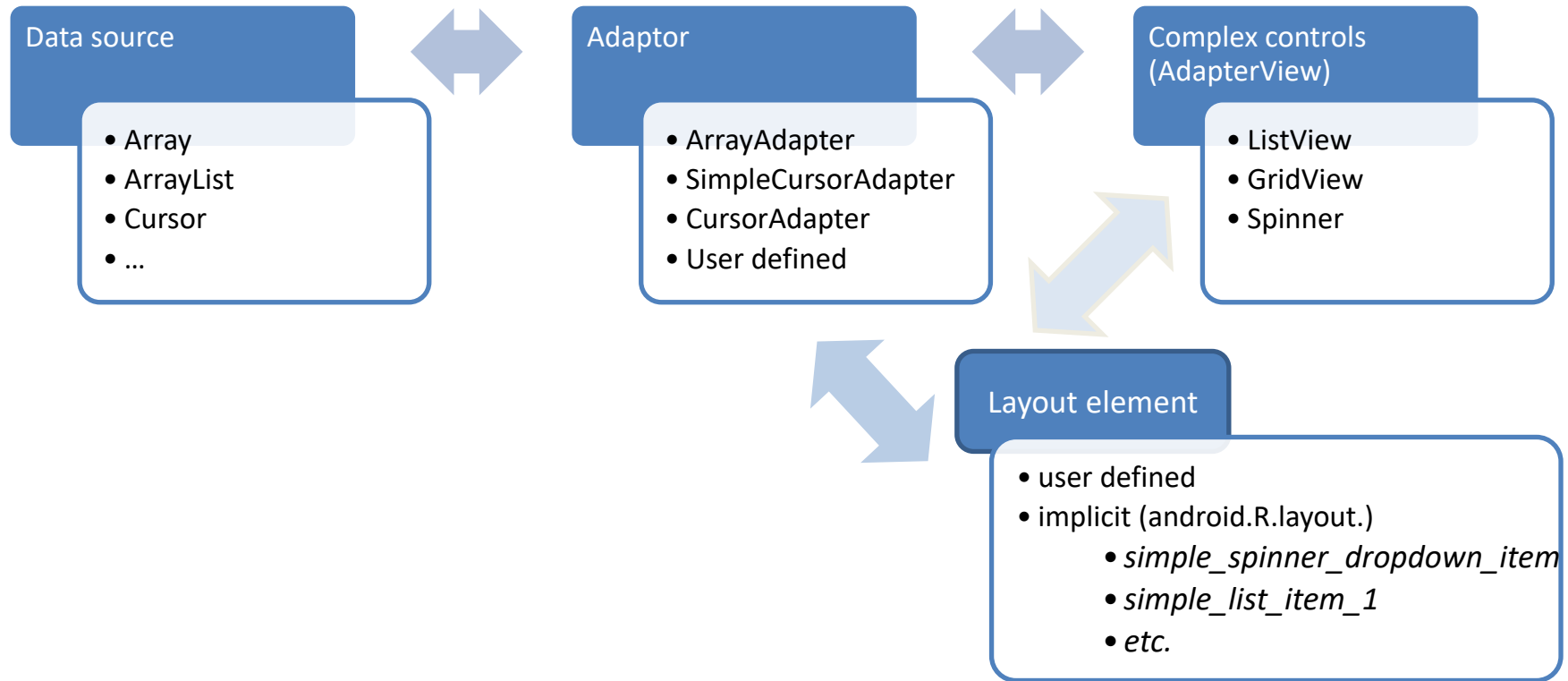
ListActivity

- `getListView()`
- `setListAdapter()`

ListFragment

RecyclerView

Complex visual controls



Adapters

Used to manage the connection between a data source and the elements of a collection

Used by the list controls for displaying data from data source

Creates a View for each element from the data source

Predefined views for list controls

Predefined resource	Description
android.R.layout.simple_spinner_item	Single line text
android.R.layout.simple_spinner_dropdown_item	Single line text with selection support
android.R.layout.simple_list_item_1	Single line text
android.R.layout.simple_list_item_2	Text displayed with two lines
android.R.layout.simple_list_item_simple_choice	Single line text with exclusive selection support
android.R.layout.simple_list_item_multiple_choice	Single line text with support for multiple selection
android.R.layout.simple_expandable_list_item_1	Single line text
android.R.layout.simple_expandable_list_item_2	Two lines text

ArrayAdapter

Uses a simple data source like:

- ArrayList, Array, etc.

Constructor:

- Context (activity)
- The resource's id for the layout associated with a list element
- The TextView identifier
- The array list of values

For each element in the list:

- data must be associated with the control used for displaying the values

Example (1)

```
String [] countries = new String[] {"Romania", "Bulgaria", "Greece"};
```

```
//list initialization
```

```
ListView lv = (ListView)findViewById(R.id.lista1);
```

```
//adaptor initialization
```

```
ArrayAdapter adaptor = new ArrayAdapter<String>(this,  
android.R.layout.simple_list_item_1, countries)
```

```
//adaptor binding with controls
```

```
lv.setAdapter(adaptor);
```

Example (2)

```
class Country{  
    String name;  
    int population;  
    Bitmap flag;  
    public String toString() {  
        return "Name: " + name + ", population: " + population;  
    }  
}
```

Example (2)

```
ArrayList<Country> countryList = new ArrayList<Country>();
```

```
//list initialization
```

```
ListView lv = (ListView)findViewById(R.id.list1);
```

```
//adaptor initialization
```

```
ArrayAdapter adaptor = new ArrayAdapter<Country>(  
    this,  
    android.R.layout.simple_list_item_1,  
    countryList);
```

```
//binding adaptor
```

```
lv.setAdapter(adaptor);
```

SimpleAdapter

- Current context;
- Object list used as data source:
 - Objects are stored using a **Map** structure
 - A String type key used for storing the values associated in columns param
- Resource identifier for each item in the layout;
- Columns param that will be displayed on each item in the list;
- The resource identifiers associated with each property, attribute;

Example (3)

```
ArrayList<Map<String, String>> countries = new ArrayList<Map<String, String>>();
```

```
final String NAME = "name";
```

```
final String POPULATION = "population";
```

```
for (Country object : countryList) {
```

```
    HashMap<String, String> country = new HashMap<String, String>();
```

```
    country.put(NAME, object.getName());
```

```
    country.put(POPULATION, object.getPopulation());
```

```
    countries.add(country );
```

```
}
```

Example (3)

```
String[] attributes= { NAME, POPULATION };  
int[] identifiers = { android.R.id.text1, android.R.id.text2 };  
  
SimpleAdapter adaptor = new SimpleAdapter(  
    this, // context  
    countries, // object list  
    android.R.layout.simple_list_item_2,  
    attributes, // the list of columns the collection should map onto  
    identifiers);  
list.setAdapter(adaptor);
```

Extending BaseAdapter class

`int getCount( )`

- Returns the number of available elements in the data source

`Object getItem(int position)`

- Returns the item from a specified position

`long getItemId(int position)`

- Returns the identifier from a specified position

`View getView(int position, View convertView, ViewGroup parent)`

- Returns a view object for displaying it in the list at a given location

Example (4)

Container

	Romania	TextView
	19.544.000 people	TextView

ImageView

res/layout/element_lista.xml

Example (4)

```
public class CountryAdaptor extends BaseAdapter {  
  
    private static class ViewHolder {  
        public ImageView flag;  
        public TextView name, population;  
    }  
  
    List<Country> countries; LayoutInflater layoutInflater;  
  
    CountryAdaptor(Context context, List<Country> countries) {  
        layoutInflater = LayoutInflater.from(context);  
        this.countries = countries;  
    }  
}
```

Example (4)

```
@Override  
public int getCount() {  
    return countries.size();  
}
```

```
@Override  
public Object getItem(int position) {  
    return countries.get(position);  
}
```

```
@Override  
public long getItemId(int position) {  
    return position;  
}
```

Example (4)

@Override

```
public View getView(int position, View convertView, ViewGroup parent) {  
    if (convertView == null) {  
        convertView = inflater.inflate(R.layout.element_list, parent, false);  
    }  
    //initializing controls found on the layout  
    ImageView flag = (ImageView) convertView.findViewById(R.id.imageViewFlag);  
    TextView name = (TextView) convertView.findViewById(R.id.textViewName);  
    TextView population = (TextView) convertView.findViewById(R.id.textViewPopulation);  
    Country country = countries.get(position);  
    //setting the values  
    flag.setImageBitmap(country.getFlag());  
    name.setText(country.getName());  
    population.setText(country.getPopulation() + " people");  
    return convertView;  
}  
}
```

Example (4)

@Override

```
public View getView(int position, View convertView, ViewGroup parent) {  
    ViewHolder holder;  
    if(convertView == null) { //the control is rendered for the first time  
        convertView = inflater.inflate(R.layout.element_list, parent, false);  
        holder = new ViewHolder();  
        holder.flag = (ImageView) convertView.findViewById(R.id.imageViewFlag);  
        holder.name = (TextView) convertView.findViewById(R.id.textViewName);  
        holder.population = (TextView) convertView.findViewById(R.id.textViewPopulation);  
        convertView.setTag(holder);  
    }  
    else { holder = (ViewHolder)convertView.getTag(); }  
    Country country = countries.get(position);  
    holder.flag.setImageBitmap(country.getFlag());  
    holder.name.setText(country.getName());  
    holder.population.setText(country.getPopulation() + " people")  
    return convertView;  
}
```

Example (4)

```
CountryAdaptor countryAdapter = new CountryAdaptor(  
    this, countryList);  
lv.setAdapter(countryAdapter);
```

Lists

Spinner

- `getSelectedItem()`
- `getSelectedItemId()`

ListView

- `getItemAtPosition()`
- `getItemIdAtPosition()`

GridView

- `getItemAtPosition()`
- `getItemIdAtPosition()`

List events

Interface	Main event method	Registering event
AdapterView.OnItemSelectedListener	onItemSelected() onNothingSelected()	setOnItemSelectedListener()
AdapterView.OnItemClickListener	onItemClick()	setOnItemClickListener()

OnItemClickListener (List)

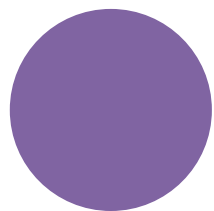
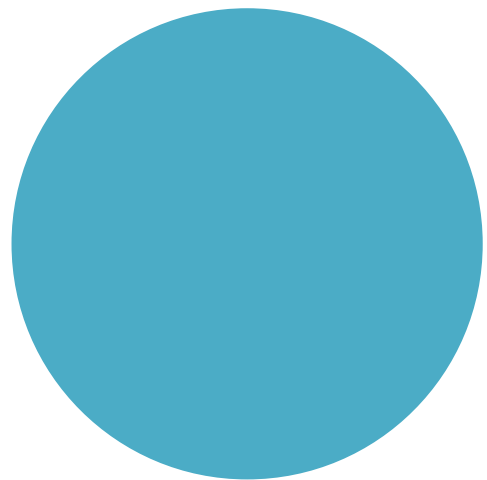
```
private AdapterView.OnItemClickListener movieLVClickListener = new
AdapterView.OnItemClickListener() {
    @Override
    public void onItemClick(AdapterView<?> parent, View view, int
position, long id) {
        String value = ((TextView)view).getText().toString();
        Toast.makeText(MovieList.this, "You selected: " +
value, Toast.LENGTH_SHORT).show();
    }
};
```

OnItemSelectedListener (Spinner)

```
spinner.setOnItemSelectedListener(new  
AdapterView.OnItemSelectedListener() {  
    @Override  
    public void onItemSelected(AdapterView<?> parent, View view, int  
position, long id) {  
        TextView tv = (TextView)view;  
        movieGenre = tv.getText().toString();  
    }  
  
    @Override  
    public void onNothingSelected(AdapterView<?> parent) {  
  
    }  
});
```

Item selection (GridView)

```
private AdapterView.OnItemClickListener gvClickListener = new
AdapterView.OnItemClickListener() {
    @Override
    public void onItemClick(AdapterView<?> parent, View view, int
position, long id) {
        TextView tv =
        (TextView) view.findViewById(R.id.textViewTitle);
        Toast.makeText(MovieGallery.this, "You clicked: " +
tv.getText(), Toast.LENGTH_SHORT).show();
    }
};
```



OBJECT SERIALIZATION



SERIALIZING OBJECTS

Serializable

- Java
- Storage
- Versioning
- Simplicity vs. fast processing
 - reflection

Parcelable

- Android
- Transport
 - Process communication
- Parcel
- Efficiency vs. complexity

Serializable Interface

- java.io
- **ObjectOutputStream**
- **ObjectInputStream**
- transient
 - the fields that you don't wish to serialize
- Interface methods
 - void **writeObject**(ObjectOutputStream out)
 - void **readObject**(ObjectInputStream in)

Parcel Class

- Data container
- Used for carrying data between processes
- Primitives & String
 - writeByte(), writeDouble(), writeString() etc.
 - readByte(), readDouble(), readString() etc.
- Complex data types
 - writeList(), writeBundle(), writeStringArray() etc.
 - readList(), readBundle(), readStringArray() etc.

Parcelable Interface

- android.os
- **writeToParcel()** - flattens the object into a Parcel
 - the object is written in a Parcel
- the **Constructor** receives a **Parcel** object
 - initializes the object from the Parcel
- **describeContents()** - describe the kinds of special objects contained in this Parcelable instance's marshaled representation
- the **static** field **CREATOR**
 - implements the Parcelable interface
 - calls for the constructor with a Parcel object

Parcelable Interface

- void **writeToParcel**(Parcel dest, int flags)
- **ClassName**(Parcel in)
- public static final Parcelable.**Creator**< **ClassName**> **CREATOR** =
new Parcelable.Creator< **ClassName**>() {
 public Clasa **createFromParcel**(Parcel pc) {
 return new **ClassName**(pc);
 }
 public Clasa[] **newArray**(int dim) {
 return new **ClassName**[dim];
 }
};
- public int **describeContents**()

Resources

- C. Boja, C. Ciurea, M. Doinea – Android Mobile Applications: A Practical Development Guide, ASE Press, 2015, 404 pp., ISBN 978-606-34-0033-9
- S. Komatineni, D. MacLean – Pro Android 4, Apress, 2012
- R. Meier – Professional Android 4 Application Development, Wiley, 2012
- P. Pocatilu, Programarea dispozitivelor mobile, Editura ASE, 2012
- P. Pocatilu, I. Ivan ș.a. – Programarea aplicațiilor Android, Editura, ASE, 2015
- <http://developer.android.com>